

一、系统总体设计

1.1 系统功能需求分析

本智能机器人面向轨道交通基础设施和环境中的巡检任务，需在轨道沿线或隧道内自主运行，完成环境参数监测与视觉数据采集。结合应用场景，系统功能需求划分为以下三个方面：

1、环境监测需求：实时检测轨道前方存在的异物、人员或设备，用于隧道或高架段环境安全评估；

2、视觉数据采集：搭载 RGB 相机和激光雷达，用于轨道扣件、道床、接触网等部位的视觉检查，采集数据可本地存储，并支持通过无线网络上传至地面服务器，支持在无定位信号的隧道环境中进行即时定位与稀疏/稠密地图构建，为巡检路径规划提供位姿参考。

3、自主避障和路径规划：基于多传感融合结果，实现局部路径重规划，避开临时障碍物后恢复原任务，支持全局路径预设，如沿轨道方向，以及局部动态行为调整，如避障、调头。

1.2 总体架构设计

采用树莓派（上位机）+ STM32（下位机）的双机异构架构，发挥各自优势：树莓派负责复杂计算（SLAM、路径规划、图像处理），STM32 负责实时底层控制（电机驱动、传感器数据采集与预处理、通信协议解析）。架构框图如下：

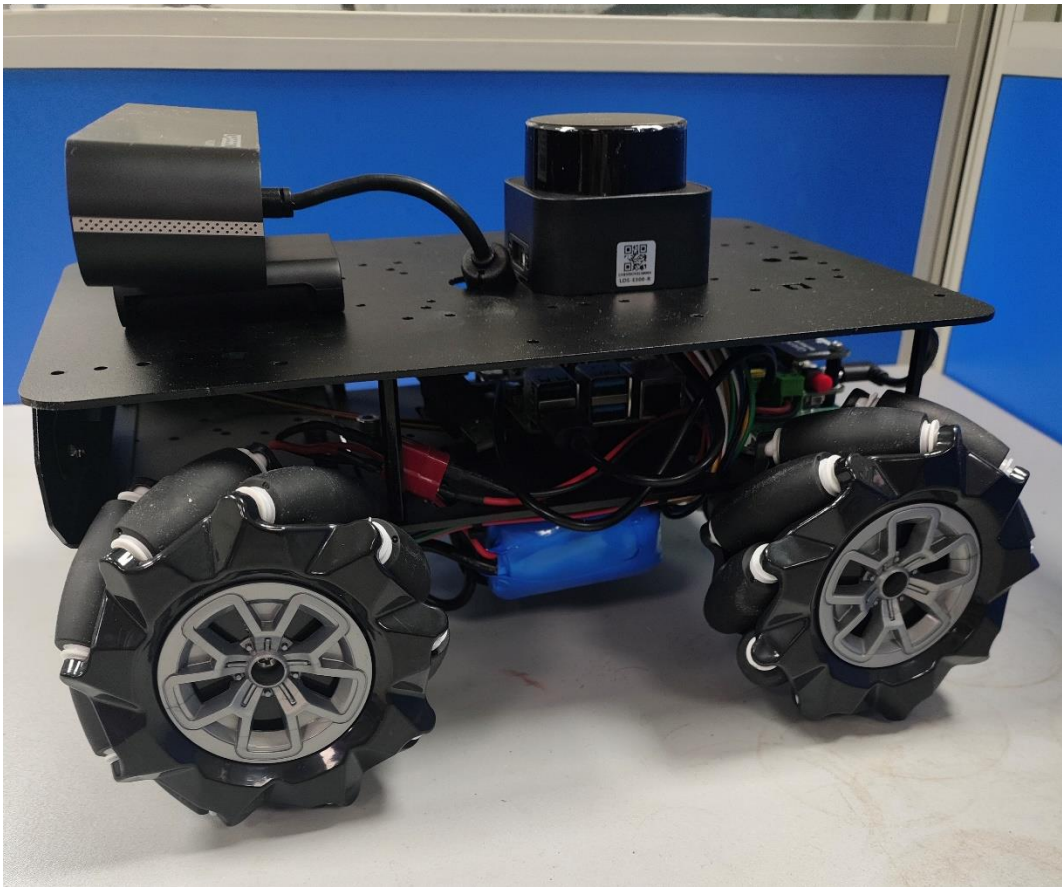


图 1 智能车结构图

本智能车基于 OpenCTR H60 控制器，是一款为专用于机器人开发的开源控制器，采用 STM32F407 作为主控制，板载 IMU 加速度陀螺仪传感器，支持四路编码器直流电机，板载两路 5V 电源，最大电流 5A，具有丰富的板载资源和扩展接口，非常适合 ROS 机器人开发，可以和树莓派、Jetson 系列、工控机等 ROS 主控组成 ROS 机器人。控制器的机器人硬件连接系统框图如下所示：

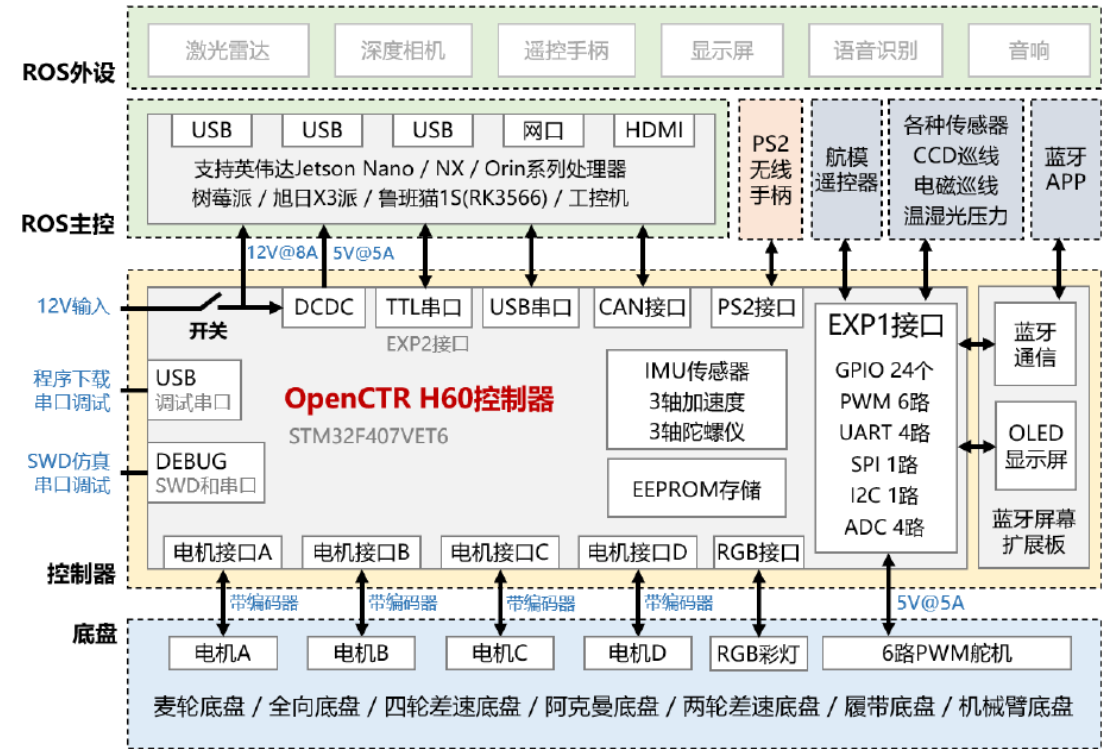


图 2 OpenCTR H60 硬件连接系统框图

本智能车基于树莓派 4B 作为上位机，其是为学生计算机编程教育而设计，只有信用卡大小的卡片式电脑，其系统基于 Linux。

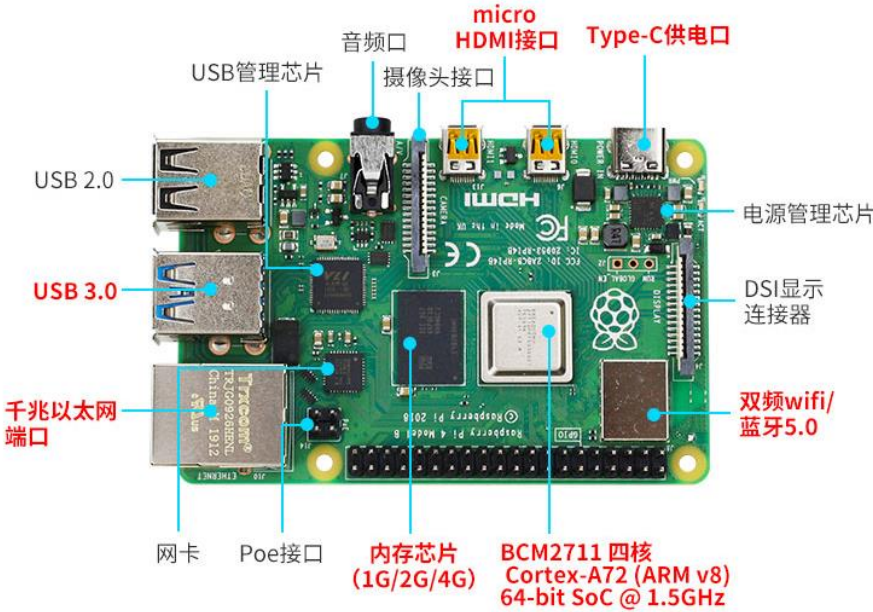


图 3 树莓派 4B 结构图

本车采用的摄像头为 Astra Pro,, 是基于 3D 结构光技术所设计的一种高精度、低功耗的 3D 摄像头, 适用于使用距离为 60cm—800cm, 可实现实时三维信息采集, 为智能终端加上了物体感知功能。



图 4 Astra Pro 摄像头

本车使用的激光雷达为 LDS—E300—R, 可实现在二维平面的 25 米半径范围内可靠测量典型旋转频率为 15 HZ(900RPM)角度分辨率为 0.3 度, 每秒高达 18000 次的测距。测距数据通过高速光通讯发送到供电处理模块进行解算, 将目标物体与雷达的距离值、强度信息从通讯接口中输出。在工作状态下, 激光器向外发射出一束激光, 照射到障碍物体上会发生反射, 接收器对反光信号进行探测, 通过时间分析模块测量出反射光与发射光之间的时间差, 用时间乘以光速即可得到光的飞行距离, 从而计算出障碍物的位置信息。为了获取更多的角度上的目标信息, 雷达内部通过电机旋转得到不同的角度上的距离和强度信息, 从而获得完整的二维点云图。



图 5 LDS—E300—R 激光雷达

二、系统使用教程

机器人底盘支持多种控制方式和控制接口，支持常规的 PS2 手柄控制，蓝牙 APP 控制，SBUS 航模遥控器控制。支持 ROS 控制，可通过 USB 串口、TTL 串口、CAN 总线连接 ROS 主控。

常规控制无需启动 ROS 系统，到手即可遥控，包括 PS2 手柄控制、蓝牙 APP 控制、SBUS 航模遥控器。

ROS 控制有串口和 CAN 控制。串口控制：机器人底盘接收来自串口的控制指令，并通过串口发送底盘的里程计、IMU、电量数据，同时支持 USB 串口和 TTL 串口。CAN：机器人底盘接收来自 CAN 口的控制指令，并通过 CAN 口发送底盘的里程计、IMU、电量数据。

2.1 机器人开关机说明

机器人打开电源开关启动，启动时有蜂鸣器提示。



机器人是一个运行 Linux 系统的电脑，关机操作不能关闭电源开关，通过 Poweroff 命令关闭机器人 Linux 系统，等待系统关闭后，再关闭电源开关。

2.2 手柄控制

机器人可通过 PS2 手柄控制，首先确保手柄接收器已经连接到 OpenCTR 控制器。接收器的红灯常亮，绿灯闪烁为等待连接状态。此时打开 PS2 手柄开关，PS2 手柄会自动连接接收器，连接成功后，接收器的绿灯为常亮状态。





右侧摇杆控制机器人 X 和 Y 速度，即摇杆上推前进，下推后退，左右控制机器人横向移动，左侧摇杆左右移动控制机器人转向，上下移动无效。同时左右摇杆也是按键，可以调节机器人速度，按“左侧摇杆按键”机器人速度减小，速度减小到最小时不能再减小，并有蜂鸣器鸣叫提示。按“右侧摇杆按键”速度增大，增大到最大时不能再增加，并有蜂鸣器鸣叫提示。

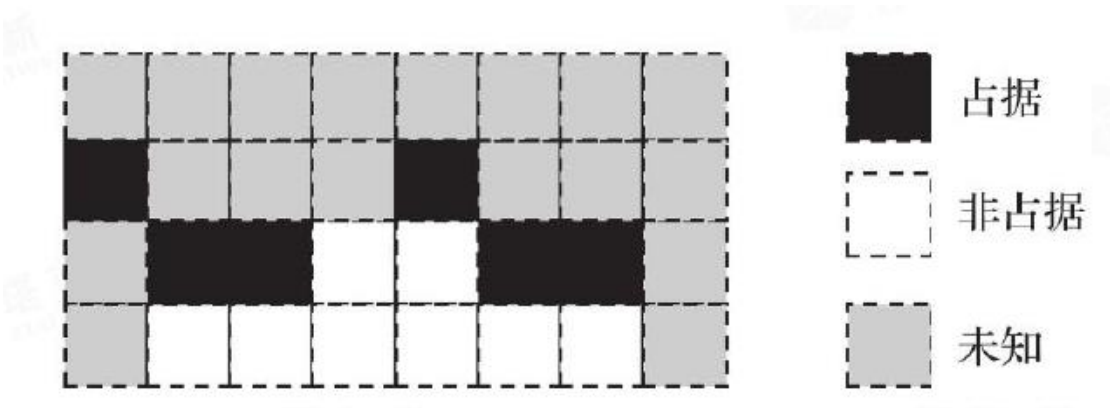
三、系统建图与导航

3.1 SLAM 建图

SLAM, 英文全称 simultaneous localization and mapping, 中文解释为即时定位与地图构建, 或并发建图与定位。SLAM 可以描述为: 机器人在未知的环境中从一个未知位置开始移动, 移动过程中根据位置估计和地图进行自身定位, 同时建造增量式地图, 实现机器人的自主定位和导航。

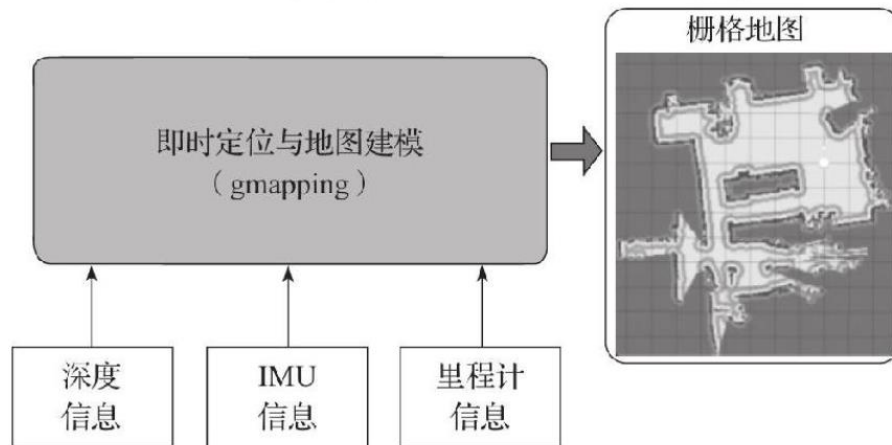
环境建模其实就是对环境状态进行描述, 也就是构建环境地图。地图可以用于定位, 也可以用于避障, 因此定位用到的地图与避障用到的地图并不一定相同。环境地图有多种, 激光 SLAM 则以构建栅格地图为主。

二维栅格地图比较简单, 就是将二维连续空间用栅格进行离散划分。机器人通常采用二维占据栅格地图, 其是对划分出来的每个栅格用一个占据概率值进行量化。如下图所示, 概率为 1 的栅格被标记为占据状态, 概率为 0 的栅格被标记为非占据状态, 概率在 0 到 1 之间的栅格被标记为未知状态。机器人在导航过程中, 要避开占据状态的栅格, 在非占据状态的栅格中通行。它通过传感器来探明未知状态的栅格的状态。



本系统机器人建图功能包为 `tarkbot_slam`, 已默认安装了常用的四种 SLAM 建图算法, 通过功能包中的 `tarkbot_map.launch` 文件启动不同的建图算法。首先启动机器人 `bringup.launch` 文件, 该文件启动了机器人底盘, 模型文件, TF 变换, 雷达等节点, 然后根据参数调用不同的算法启动文件, 默认为 `Gmapping` 算法。

`gmapping` 功能包集成了 Rao-Blackwellized 粒子滤波算法, 为开发者隐去了复杂的内部实现。`gmapping` 功能包的总体框架如下图。



3.2 建图操作

首先 SSH 连接到机器人端，运行 `tarkbot_slam` 功能包中 `map.launch` 启动文件，启动

`gmapping` 建图节点。

`roslaunch tarkbot slam map.launch slam_methods:=gmapping`

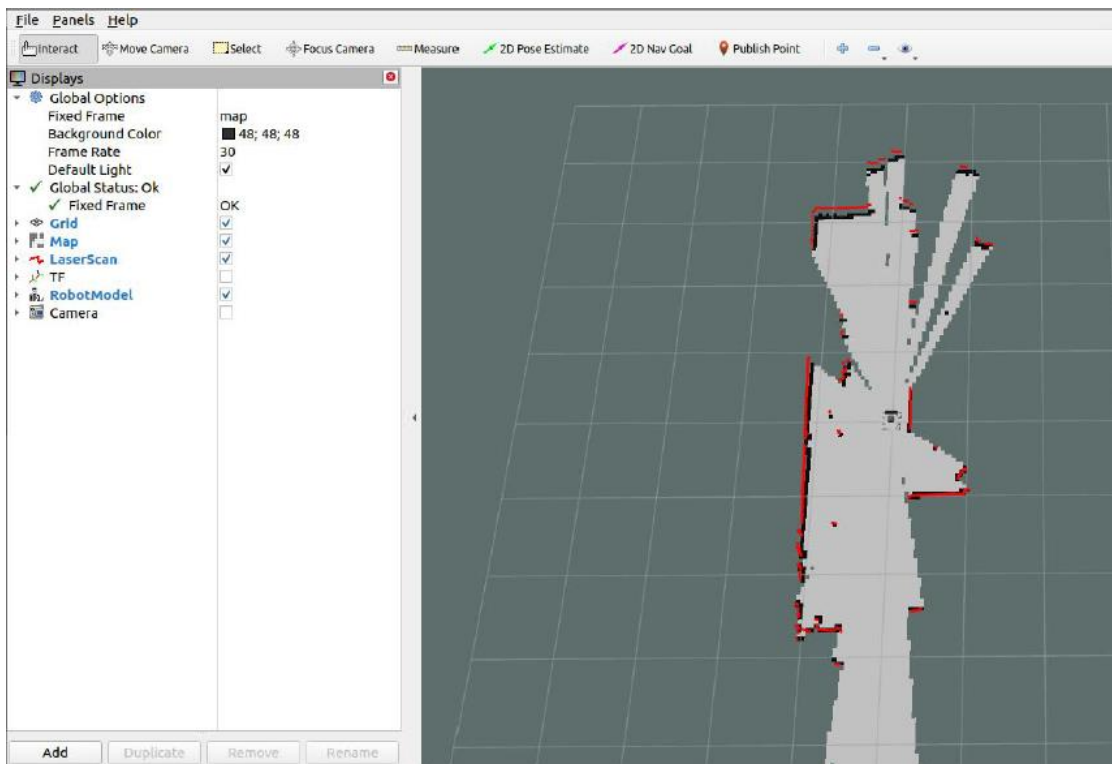
在虚拟机中运行 `rviz` 可视化工具：

`roslaunch rviz rviz`

启动后在 `rviz` 工具中添加虚拟机的配置文件，点击 `rviz` 左上角的 `file/Open Config`，在图示路径下找到 `rviz` 文件，最后双击加载后将看到如下画面。



选择 `Close without Saving` 弹出建图界面。



运行键盘控制节点，控制机器人移动建图，也可以使用手柄控制移动

```
roslaunch tarkbot teleop keyboard.launch
```

建图完成后，此时地图数据是保存在内存中的，当节点关闭时，数据也会被一并释放，需要将栅格地图保存到磁盘中，以备机器人导航使用。地图保存可以通过 `map_server` 功能包实现。

首先 SSH 连接到机器人端，运行如下 `launch` 文件，地图将保存在 `tarkbot slam` 功能包的 `maps` 文件加下，文件名可自定义。

```
roslaunch tarkbot slam save.launch name:=xxx
```

保存成功后，在 `maps` 文件夹下会增加 `xxx.pgm` 和 `xxx.yaml` 两个文件，这就是保存的地图文件。

本车还针对不同传感器和环境下，内置了多种建图方法，包括 Hector 建图、Karto 建图、Cartographer 建图。使用方式同 Gmapping 建图。

3.3 导航操作

导航就是解决从地点 A 到地点 B 的问题，通俗地讲自主导航需要解决机器人“我在哪里”“我要到哪里去”以及“我该如何过去”这三大问题。

机器人已完成建图后，并且对二维栅格地图有一定了解，在使用构建的地图进行自主导航运行，SLAM 自主导航流程，先手动遥控机器人进行 SLAM 环境扫描并将构建好的地图保存下来，然后载入事先构建好的离线地图并启动 SLAM 重定位模式获取机器人的实时位姿。再指定机器人目标点，机器人通过自身雷达等传感器进行定位换环境检测，通过规划的路径导致指定地点，期间遇到障碍物也可以动态躲避。在进行机器人导航之前需要构建地图，并保存地图。

运行 `tarkbot_nav` 功能包前，SSH 连接到机器人端，执行如下指令修改 `nav.launch` 文件中的地图文件路径和名称。

```
roscd tarkbot_nav/launch/
vi nav.launch
```



```

1 <launch>
2
3 <!-- 标签 -->
4 <arg name="map_file" default="multimap.yaml" /> 地图文件名称
5
6 <!-- 启动机器人，包含底盘，雷达，相机设备 -->
7 <include file="$(find tarkbot_bringup)/launch/bringup.launch" >
8   <arg name="use_pose_ekf" value="true"/>
9   <arg name="use_lidar" value="true"/>
10  <arg name="use_camera" value="false"/>
11  <arg name="use_web_video" value="false"/>
12 </include>
13
14 <!-- 启动Map server功能包，发布地图 --> 地图文件路径
15 <node name="map_server" pkg="map_server" type="map_server" args="$(find tarkbot_slam)/maps/${arg
   map_file}" />

```

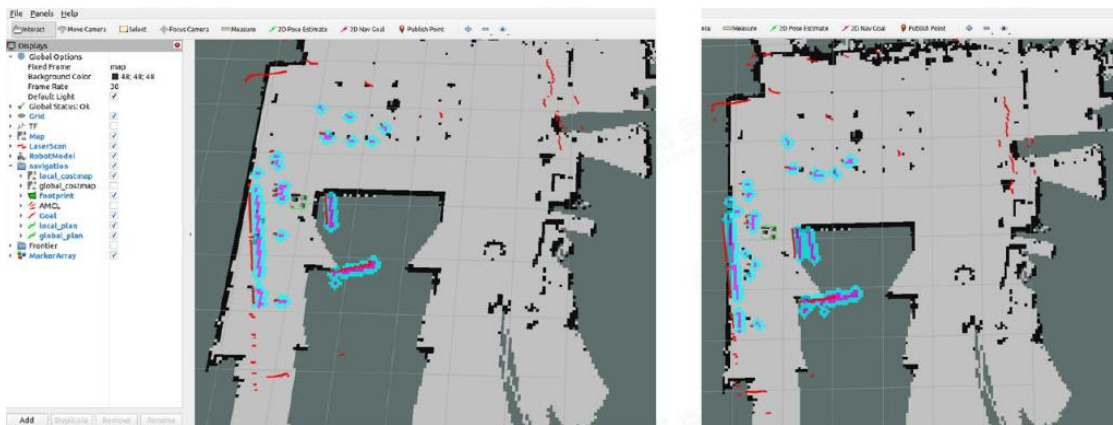
保存后启动该 launch 文件时显示地图就更换为当前修改的地图了。

直接连接到机器人，在机器人端运行 tarkbot_nav 导航驱动包的导航节点，会在终端输出启动信息。

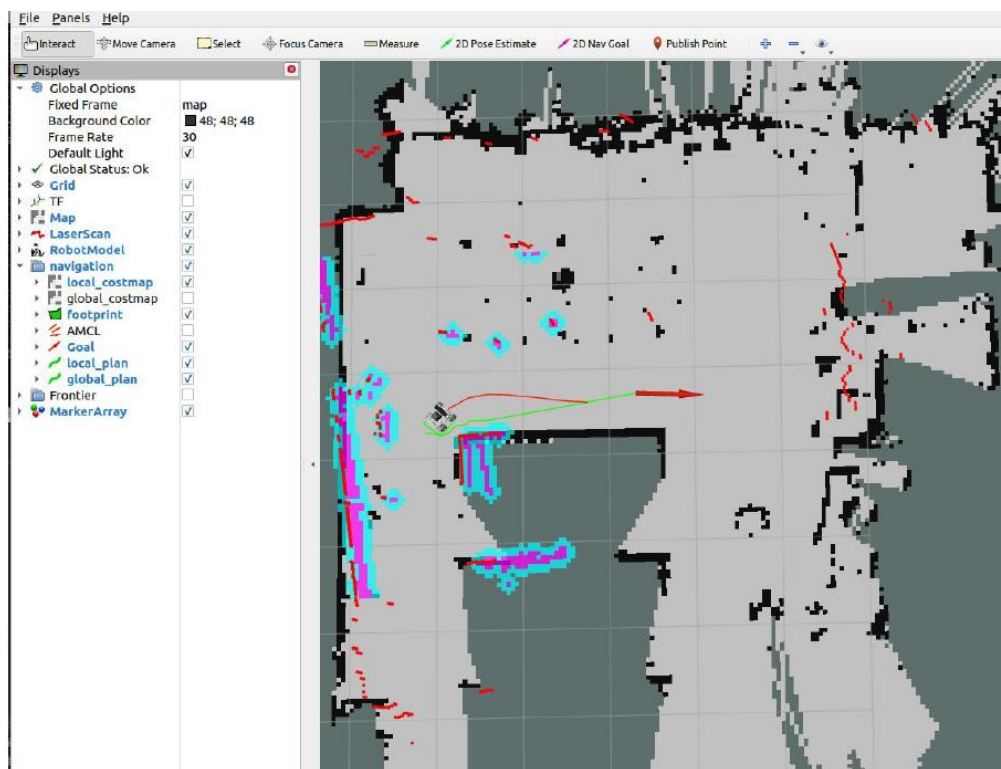
```
roslaunch tarkbot_nav nav.launch
```

在虚拟机端运行 rviz 工具，启动后在 rviz 工具中添加虚拟机的配置文件，点击 rviz 左上角的 file/Open Config，找到 rviz 文件，弹出建图界面。

导航前首先进行机器人初始位置标定，根据下图实际场景中机器人的位置和朝向标定。使用 RVIZ 的“2DPose Estimate”按钮进行机器人标定，箭头的尾部为 ROS 机器人的位置，箭头的方向为 ROS 机器人的朝向，如下图所示。



标定完初始位置后进行导航，初始位置大体准确即可，即使有少量偏差，AMCL 算法也会调整正确。使用 RVIZ 的“2DNav Goal”按钮发布一个目标位置，ROS 机器人即可自动导航与避障到达目标位置。导航过程中，如果有人遮挡机器人前进，机器人也会重新规划路径自动避让。如图所示其中绿色为全局路径，红色为局部路径。



借助 rqt 可视化工具, 可以查看导航过程中 ROS 节点之间的数据流向以及 tf 状态。导航过程中 ROS 节点之间的数据流向查看命令如下所示。

```
roslaunch rqt rqt_graph
```

四、障碍物识别

轨道交通场景中常见的障碍物包括：行人、工具、施工锥桶、落石、动物等，形态多样且背景复杂（隧道低光照、轨道反光）。传统基于手工特征（HOG+ SVM）的方法泛化能力不足。YOLOv5 作为单阶段目标检测算法，在精度与实时性之间取得良好平衡，适合部署在树莓派等边缘设备上。YOLOv5 由以下四部分组成：

1、输入端：Mosaic 数据增强、自适应锚框计算、自适应图片缩放

2、Backbone：CSPDarknet53（Cross Stage Partial Network），采用 CSP 结构将梯度变化从头到尾合并，减少计算量；使用 Focus 模块对输入图像进行切片操作，降低参数量。

3、Neck：PANet（Path Aggregation Network）结构，自顶向下与自底向上的特征金字塔，融合多尺度特征，增强小目标检测能力。

4、Head：3 个不同尺度的检测头（ 80×80 ， 40×40 ， 20×20 ），分别负责小、中、大目标，输出类别概率、置信度及边界框。

由于模型需要提前训练完成，将模型转换为 ONNX 格式，再使用 TensorRT 或 Tengine 进行 FP16 量化推理，输入分辨率压缩为 512×512 。为了方便演示，本说明使用官方训练好的模型。

首先需要使用 VNC 连接到机器人，若发现连接后分辨率不适合操作，可执行如下指令更换分辨率。

```
xrandr --fb 512x512
```

分辨率可根据需要进行调节，不过需要考虑网络带宽等问题，不建议设置太大。之后可以直接执行如下指令，查看运行效果。

```
python3 detect.py
```

```
cd ~/tarkbot/depth_learning/yolov5
```

本机器人当前识别图片为官方提供的两张图片，图片的保存位置为 /yolov5/data/images 文件夹下，如果有其它需要识别的图片也可以放入该文件夹下。

识别完成后会显示一下然后图片自动关闭，识别后的标记图片将会保存到以下目录 /yolov5/runs/detect/exp



再结合上我们的 Astra Pro 相机，可以实时识别到物体：

```
cd ~/tarkbot/depth_learning/yolov5
```

```
python3 video_detect.py
```

初始加载过程会比较缓慢，待 RGB 图像出现后其就开始识别了，识别效果如

下：

